Лабораторна робота № 3

Тема. Застосування ітераторів, генераторів для ефективного опрацювання даних.

Мета: навчитися використовувати ітератори та генератори для ефективного опрацювання даних.

Короткі теоретичні відомості

Ітератор – це об’єкт, який забезпечує механізм послідовного перебору елементів деякої колекції (або потоку даних), запам’ятовуючи позицію поточного елемента. Він дозволяє отримувати наступний елемент лише тоді, коли це потрібно (ліниві обчислення), без необхідності завантажувати всі дані в пам’ять одночасно. Щоб об’єкт вважався ітератором у Python, він повинен реалізувати два спеціальні методи: *__iter__()* – повертає сам об’єкт ітератор; *__next__()* – повертає наступний елемент. Якщо об’єкт реалізує лише *__iter__()*, але не реалізує *__next__()*, він називається ітерованим, але не ітератором. Приклади ітерованих об’єктів: *list*, *tuple*, *str*, *dict*, *set*, *range*. Приклад того як працює цикл *for* «під капотом»:

```
ages = [10, 18, 25]
iterator = iter(ages) # викликає obj.__iter__()
print(next(iterator)) # викликає iterator.__next__()
print(next(iterator))
print(next(iterator))
```

Генератор – це спеціальний тип ітератора в Python, який створюється за допомогою звичайної функції з ключовим словом *yield* замість *return*. Він дозволяє призупиняти виконання функції, зберігаючи її стан, і відновлювати його при наступному запиті на отримання елемента.

yield – призупиняє функцію, повертає значення, але зберігає всі локальні змінні та поточну позицію виконання.

Приклад створення генератора за допомогою функції з *yield*:

```
def count_down(n):
  while n >= 0:
    yield n
    n -= 1
for num in count_down(5):
  print(num)
```

Спискове включення (list comprehension) – це лаконічний та ефективний спосіб створення списків у Python. Приклад:

```
# залишимо лише парні числа
evens = [x for x in range(20) if x % 2 == 0]
```

Приклад створення генераторного виразу:

```
# Списковe включення (створює список у пам'яті одразу)
squares_list = [x**2 for x in range(5)]
# Генераторний вираз (створює об'єкт генератора, обчислює ліниво)
squares_gen = (x**2 for x in range(5))
print(next(squares_gen))
print(next(squares_gen))
```

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.
2. Створити власні генератори за допомогою функції з *yield* та генераторного виразу відповідно до варіанта та використати їх.
3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Опис генератора (що саме повинен повертати)
1	Числа Фібоначчі до N, але тільки ті, що діляться на 3
2	Факторіали чисел від 1 до N
3	Сума цифр кожного числа від 1 до N
4	Числа, що містять цифру 7
5	Зворотні числа (зворотній порядок цифр)
6	Числа кратні 3 або 5 (до N)
7	Непарні числа від 0 до N

Контрольні питання

1. Що таке ітератор у Python?
2. Яка головна відмінність між генератором та звичайною функцією?
3. Назвіть два способи створення генератора.
4. Як отримати наступний елемента з генератора?
5. Що таке ітерований об’єкт і чим він відрізняється від ітератора?